

OPTA-PROG-SYNCHRONIZATION-VARIABLES-CLOUD

PT-BR

Sincronização dos valores salvos na Cloud com uma variável Offline

1. Informações Gerais

- **Versão / Revisão:** *Revisão 1*
- **Data de Desenvolvimento:** 29/09/2024
- **Autor:** Daniel Arcos
- **Linguagem:** C++
- **Plataforma / Hardware:** Finder OPTA

2. Objetivo da Programação

A programação tem como objetivo demonstrar como realizar uma contagem simples em uma das entradas do OPTA e disponibilizar esse valor na Cloud da Arduino. Além disso, a programação sincroniza o último valor lido salvo pela Cloud com a variável offline. Dessa forma a contagem irá continuar a partir do último valor recebido pela Cloud.

3. Requisitos e Dependências

- Instalar os drivers de Hardware do OPTA no Arduino IDE
- Instalar biblioteca do OPTA
- Criar conta na plataforma Arduino IoT Cloud, pois ela será utilizada para demonstrar a contagem.

4. Passo a Passo da Implementação

Conexões Físicas:

1. Conecte o sensor de pulsos ao pino A0 (podendo ser um pulsador ou interruptor para simulação)
2. Conecte a carga controlada ao pino D0 do OPTA (caso queira, mas não é necessário)
3. Conecte o OPTA ao computador via USB para programação

Configuração do Software:

1. Acesse o Arduino IoT Cloud e crie uma nova "Thing"
2. Adicione as variáveis conforme especificado no código
3. Gere o código base e substitua pelo código fornecido
4. Adicione as bibliotecas necessárias através do Arduino IDE
5. Faça o upload do código para o OPTA

5. Estrutura Geral do Código

Includes e Variáveis Globais:

```
#include "thingProperties.h"
```

```
int estadoentrada1 = 0;  
double contagem = 3.928;  
bool conmuta = true;
```

- Inclui automaticamente as definições da IoT Cloud
- Declara variáveis locais para controle do estado
- Inicializa a contagem de consumo em 3.928 kWh

Função setup (Configuração Inicial):

```

void setup() {
  Serial.begin(9600);
  delay(1500); // Espera pelo Serial Monitor

  initProperties(); // Inicializa propriedades da nuvem
  ArduinoCloud.begin(ArduinoIoTPreferredConnection); // Conecta à nuvem

  setDebugMessageLevel(2); // Define nível de debug
  ArduinoCloud.printDebugInfo(); // Mostra informações de conexão

  // Adiciona callbacks para eventos de sincronização e desconexão
  ArduinoCloud.addCallback(ArduinoIoTCloudEvent::SYNC, doThisOnSync);
  ArduinoCloud.addCallback(ArduinoIoTCloudEvent::DISCONNECT, doThisOnDisconnect);

  // Configura os pinos
  pinMode(A0, INPUT); // Entrada do sensor de pulsos
  pinMode(D0, OUTPUT); // Saída para carga
  pinMode(LED_D0, OUTPUT); // LED interno
}

```

Função loop (Loop Principal):

```

void loop() {
  ArduinoCloud.update(); // Atualiza conexão com a nuvem

  // Leitura do sensor de pulsos
  estadoentrada1 = digitalRead(A0);
  if(estadoentrada1 == HIGH) {
    delay(5); // Debounce
    if (conmuta) {
      Serial.print("Consumo kWh: ");
      Serial.println(contagem);
      contagem = contagem + 0.002; // Incrementa consumo
      iniciado = contagem; // Atualiza variável na nuvem
    }
  }
}

```

```

    conmuta = false; // Previne múltiplas contagens
  }
}
else {
  conmuta = true; // Reseta para próxima detecção
}

// Controle da saída baseado no estado do botão na nuvem
if(btn1 == HIGH){
  digitalWrite(D0, HIGH);
  digitalWrite(LED_D0, HIGH);
}else{
  digitalWrite(D0, LOW);
  digitalWrite(LED_D0, LOW);
}
}
}

```

Funções de Callback:

```

void doThisOnSync(){
  contagem = iniciado; // Sincroniza contagem local com valor da nuvem
  Serial.println(contagem);
}

void doThisOnDisconnect(){
  Serial.println("OPTA está desconectado");
  if (ArduinoCloud.connected() == 0){
    disconnectAfter(); // Executa rotina offline
    while (false); // Placeholder para possível lógica adicional
  }
}

void disconnectAfter(){
  Serial.println("Entrando na rotina offline");
  // Continua medindo consumo mesmo offline
  estadoentrada1 = digitalRead(A0);
}

```

```

if(estadoentrada1 == HIGH) {
    delay(5);
    if (conmuta) {
        Serial.print("Consumo kWh: ");
        Serial.println(contagem);
        contagem = contagem + 0.002;
        iniciado = contagem;
        conmuta = false;
    }
}
else {
    conmuta = true;
}
}

```

Funções de Mudança de Variáveis (Automáticas):

```

void onContagemChange() {
    // Executada quando a variável 'contagem' é alterada na nuvem
}

void onBtn1Change() {
    // Executada quando o estado do 'btn1' é alterado na nuvem
}

void onIniciadoChange() {
    // Executada quando a variável 'iniciado' é alterada na nuvem
}

```

6. Estrutura Geral do Código

Este código implementa um sistema de medição de consumo de energia com as seguintes características:

1. Medição de Consumo:

- Detecta pulsos no pino A0 (simulando um medidor de energia)
- Cada pulso adiciona 0.002 kWh à contagem total
- Atualiza a variável na nuvem em tempo real

2. Controle Remoto:

- Permite controlar uma carga remotamente através da variável `btn1`
- Ativa/desativa o pino D0 e o LED interno conforme o estado do botão

3. Sincronização com Nuvem:

- Mantém os dados sincronizados entre o dispositivo e a nuvem
- Funciona mesmo durante desconexões temporárias
- Recupera o último estado ao reconectar

4. Operação Offline:

- Continua medindo o consumo mesmo sem conexão com a nuvem
- Sincroniza os dados quando a conexão é restabelecida

Este sistema é ideal para aplicações de monitoramento de energia residencial ou industrial, permitindo tanto o acompanhamento remoto do consumo quanto o controle de cargas elétricas.

EN

Synchronizing values saved in the Cloud with an Offline variable

1. General Information

- **Version / Revision:** *Revision 1*
- **Development Date:** 09/29/2024

- **Author:** Daniel Arcos
- **Language:** C++
- **Platform / Hardware:** Finder OPTA

2. Programming Objective

The purpose of this program is to demonstrate how to perform a simple count on one of the OPTA's inputs and make that value available on the Arduino Cloud. Furthermore, the program synchronizes the last read value saved by the Cloud with the offline variable. This way, the count will continue from the last value received by the Cloud.

3. Requirements and Dependencies

- Install the OPTA hardware drivers in the Arduino IDE
- Install the OPTA library
- Create an account on the Arduino IoT Cloud platform, as it will be used to demonstrate the counting process.

4. Implementation Step-by-Step

Physical Connections:

1. Connect the pulse sensor to pin A0 (it can be a pulser or a switch for simulation).
2. Connect the controlled load to pin D0 of the OPTA (if desired, but not necessary).
3. Connect the OPTA to the computer via USB for programming.

Software Setup:

1. Access the Arduino IoT Cloud and create a new "Thing."
2. Add the variables as specified in the code.

3. Generate the base code and replace it with the provided code.
4. Add the necessary libraries using the Arduino IDE.
5. Upload the code to OPTA.

5. General Code Structure

Includes and Global Variables:

```
#include "thingProperties.h"

int estadoentrada1 = 0;
double contagem = 3.928;
bool conmuta = true;
```

- Automatically includes IoT Cloud settings
- Declares local variables for state control
- Initializes consumption metering at 3,928 kWh

Setup function (Initial Configuration):

```
void setup() {
  Serial.begin(9600);
  delay(1500); // Espera pelo Serial Monitor

  initProperties(); // Inicializa propriedades da nuvem
  ArduinoCloud.begin(ArduinoIoTPreferredConnection); // Conecta à nuvem

  setDebugMessageLevel(2); // Define nível de debug
  ArduinoCloud.printDebugInfo(); // Mostra informações de conexão

  // Adiciona callbacks para eventos de sincronização e desconexão
  ArduinoCloud.addCallback(ArduinoIoTCloudEvent::SYNC, doThisOnSync);
  ArduinoCloud.addCallback(ArduinoIoTCloudEvent::DISCONNECT, doThisOnDisconnect);
}
```

```

// Configura os pinos
pinMode(A0, INPUT); // Entrada do sensor de pulsos
pinMode(D0, OUTPUT); // Saída para carga
pinMode(LED_D0, OUTPUT); // LED interno
}

```

Loop function (Main Loop):

```

void loop() {
  ArduinoCloud.update(); // Atualiza conexão com a nuvem

  // Leitura do sensor de pulsos
  estadoentrada1 = digitalRead(A0);
  if(estadoentrada1 == HIGH) {
    delay(5); // Debounce
    if (conmuta) {
      Serial.print("Consumo kWh: ");
      Serial.println(contagem);
      contagem = contagem + 0.002; // Incrementa consumo
      iniciado = contagem; // Atualiza variável na nuvem
      conmuta = false; // Previne múltiplas contagens
    }
  }
  else {
    conmuta = true; // Reseta para próxima detecção
  }

  // Controle da saída baseado no estado do botão na nuvem
  if(btn1 == HIGH){
    digitalWrite(D0, HIGH);
    digitalWrite(LED_D0, HIGH);
  }else{
    digitalWrite(D0, LOW);
    digitalWrite(LED_D0, LOW);
  }
}

```

```
}  
}
```

Callback Functions:

```
void doThisOnSync(){  
    contagem = iniciado; // Sincroniza contagem local com valor da nuvem  
    Serial.println(contagem);  
}  
  
void doThisOnDisconnect(){  
    Serial.println("OPTA está desconectado");  
    if (ArduinoCloud.connected() == 0){  
        disconnectAfter(); // Executa rotina offline  
        while (false); // Placeholder para possível lógica adicional  
    }  
}  
  
void disconnectAfter(){  
    Serial.println("Entrando na rotina offline");  
    // Continua medindo consumo mesmo offline  
    estadoentrada1 = digitalRead(A0);  
    if(estadoentrada1 == HIGH) {  
        delay(5);  
        if (conmuta) {  
            Serial.print("Consumo kWh: ");  
            Serial.println(contagem);  
            contagem = contagem + 0.002;  
            iniciado = contagem;  
            conmuta = false;  
        }  
    }  
    else {  
        conmuta = true;  
    }  
}
```

```
}  
}
```

Variable Change Functions (Automatic):

```
void onContagemChange() {  
    // Executada quando a variável 'contagem' é alterada na nuvem  
}  
  
void onBtn1Change() {  
    // Executada quando o estado do 'btn1' é alterado na nuvem  
}  
  
void onIniciadoChange() {  
    // Executada quando a variável 'iniciado' é alterada na nuvem  
}
```

6. General Code Structure

This code implements an energy consumption measurement system with the following features:

1. Consumption Measurement:

- Detects pulses on pin A0 (simulating an energy meter)
- Each pulse adds 0.002 kWh to the total count
- Updates the variable in the cloud in real time

1. Remote Control:

- Allows you to control a load remotely through the variable `btn1`
- Activates/deactivates pin D0 and the internal LED depending on the button state

1. Cloud Synchronization:

- Keeps data synchronized between the device and the cloud
- Works even during temporary disconnections
- Recovers the last state when reconnected

1. Offline Operation:

- Continues measuring consumption even without a cloud connection
- Synchronizes data when the connection is reestablished

This system is ideal for residential or industrial energy monitoring applications, allowing both remote consumption monitoring and load control electrical.